



Software Requirements Specification

For
DocuLaTeX

Version 1.2

24 August 2025

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Document Conventions	2
1.3	Intended Audience and Reading Suggestions	2
1.4	Product Scope	2
1.5	References	2
2	Overall Description	3
2.1	Product Perspective	3
2.2	Product Functions	3
2.3	User Classes and Characteristics	3
2.4	Operating Environment	3
2.5	Design and Implementation Constraints	3
2.6	Assumptions and Dependencies	4
3	Specific Requirements	4
3.1	External Interface Requirements	4
3.1.1	Command-Line Interface (CLI)	4
3.2	Functional Requirements	4
3.3	Non-Functional Requirements	5
3.3.1	Performance	5
3.3.2	Reliability	5
3.3.3	Software Quality Attributes	6
3.4	Error Handling and Recovery	6

1. Introduction

1.1 Purpose

This document defines the functional and non-functional requirements for the DocuLaTeX application. Its primary purpose is to provide a clear and unambiguous specification for a tool that converts documents from common formats (PDF, Markdown, HTML) into a themed LaTeX output.

1.2 Document Conventions

This document follows the IEEE 830-1998 standard. All requirements are uniquely identified for traceability. British International English spelling and grammar are used throughout.

1.3 Intended Audience and Reading Suggestions

This document is intended for project managers, software developers, the quality assurance team, and project stakeholders involved in the development of DocuLaTeX.

1.4 Product Scope

Product Scope

DocuLaTeX is a utility application designed to automate the process of formatting documents into a high-quality, consistent, and professional layout using a predefined LaTeX theme. It accepts input from PDF, Markdown, and HTML files and generates a compilable `.tex` file as output.

Version 1.0 (MVP) Scope: The initial release will focus on the core conversion engine. The scope includes: a command-line interface (CLI) for file input, a robust parsing module for each supported format, a mapping engine to convert parsed elements to the specified LaTeX theme, and the generation of the final `.tex` file. The MVP will handle well-structured, simple documents and will not support complex features like embedded media or scripts.

1.5 References

- IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications*.
- AUCDT Brand Identity Guide v2.0, August 2025.
- The LaTeX source of `srs.tex` for CounselorPro, which serves as the target theme definition.

2. Overall Description

2.1 Product Perspective

DocuLaTeX is a new, self-contained, standalone utility. Initially, it will be a command-line tool, with the potential for a graphical user interface (GUI) or a web-based service in future versions. It is not dependent on any other system except for a user-installed LaTeX distribution for compiling the output.

2.2 Product Functions

The major functions are: Input File Validation, Content Parsing (per format), Structural Element Mapping, LaTeX Code Generation, and Output File Creation.

2.3 User Classes and Characteristics

User Classes

Academics & Researchers: Need to quickly format papers and articles into a consistent style without deep LaTeX knowledge.

Technical Writers: Require a tool to automate the conversion of documentation from Markdown or HTML into print-ready PDFs.

Students: Want an easy way to make their reports and assignments look professional and adhere to specific formatting guidelines.

2.4 Operating Environment

The application will be a cross-platform command-line tool, capable of running on Windows, macOS, and Linux environments. It requires a functioning LaTeX distribution (e.g., TeX Live, MiKTeX) to be installed and accessible in the system's PATH for users who wish to compile the output.

2.5 Design and Implementation Constraints

📌 Important Note

The primary constraint is the fidelity of the conversion. The generated LaTeX code must be clean, readable, and must compile perfectly into a PDF that visually matches the target theme.

⚠️ **C-1 (LaTeX Dependency):** The application itself will not compile the LaTeX code. It relies on an external LaTeX distribution on the user's system.

⚠️ **C-2 (Parsing Limitations):** The parsing engines must be robust but are not expected to handle poorly-formed or exceptionally complex source files in V1.0.

For PDFs, only text-based documents are supported, not scanned images of text.

⚠ C-3 (Theme Hardcoding): The target LaTeX theme (based on the AUCDT style) is hardcoded for V1.0. Future versions may support configurable themes.

⚠ C-4 (No GUI): The V1.0 product will be a CLI-only application to prioritise the core conversion logic.

2.6 Assumptions and Dependencies

Assumption: Users have a basic understanding of how to operate a command-line tool.

Assumption: Input files are well-structured. For example, HTML uses semantic tags, and PDFs have selectable text.

Dependency: The system depends on third-party libraries for parsing PDF, Mark-down, and HTML content.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 Command-Line Interface (CLI)

The primary interface will be through the command line. Users will interact with the application by providing arguments.

CLI Specifications

The CLI shall support the following commands and arguments:

- **Input Path:** A mandatory argument specifying the path to the source file (e.g., `doculatem --input "report.pdf"`).
- **Output Path:** An optional argument to specify the directory and filename for the output `.tex` file. If omitted, it defaults to the same directory as the input file.
- **Help Command:** A `--help` flag to display usage instructions and options.

3.2 Functional Requirements

ID	Requirement Description
FR-1: File Input and Validation	
FR-1.1	The system shall accept a file path as a command-line argument.
FR-1.2	The system shall validate that the input file exists and has a supported extension (.pdf, .md, .html).
FR-2: Content Parsing	
FR-2.1	The system shall extract structural elements from Markdown files, including headers (#), lists (*, -), bold (**), italic (*), and tables.
FR-2.2	The system shall extract structural elements from HTML files, including headers (h1-h3), paragraphs (p), lists (ul, ol, li), and tables.
FR-2.3	The system shall extract text content, paragraph breaks, and basic structural hierarchy from text-based PDF files.
FR-3: LaTeX Element Mapping	
FR-3.1	The system shall map parsed headers to corresponding LaTeX sectioning commands (\section, \subsection).
FR-3.2	The system shall map parsed lists (ordered and unordered) to LaTeX list environments (itemize, enumerate).
FR-3.3	The system shall map parsed tables into the LaTeX longtable environment.
FR-3.4	The system shall escape special LaTeX characters within the text content to prevent compilation errors.
FR-4: Output Generation	
FR-4.1	The system shall generate a single, self-contained .tex file.
FR-4.2	The generated file shall include the complete preamble from the target theme, including all necessary packages, colour definitions, and custom commands.
FR-4.3	The system shall save the generated .tex file to the user-specified output path or the default location.

3.3 Non-Functional Requirements

3.3.1 Performance

NFR-1.1: The conversion of a 20-page document with mixed text and tables shall complete in under 10 seconds on a standard desktop machine.

3.3.2 Reliability

NFR-2.1: The application shall not crash or enter an infinite loop when processing valid, supported input files.

NFR-2.2: The application shall handle file I/O errors (e.g., permission denied) gracefully.

3.3.3 Software Quality Attributes

- ★ **NFR-3.1 (Correctness):** The generated `.tex` file must be syntactically correct and compile without errors using a standard LuaLaTeX engine.
- ★ **NFR-3.2 (Maintainability):** The parsing logic for each file type shall be modular and decoupled from the LaTeX generation engine to facilitate future updates.

3.4 Error Handling and Recovery

- ❗ **EH-1:** If an unsupported file type is provided, the system shall exit gracefully and display an error message listing the supported formats.
- ❗ **EH-2:** If an input file is not found or is unreadable, the system shall display a clear error message indicating the problem.
- ❗ **EH-3:** If the parsing process fails due to a malformed input file, the system shall notify the user that the file could not be processed.